



Open Chiplet Atlas Harness (OCAH)

Starting with OCAH

Table of Contents

1. Getting Started Guide	2
1.1. Dependencies	2
1.2. Workflows	3
1.3. OCAH development guidelines	5
1.3.1. Standards	5
1.3.2. Development Tools	5
1.3.3. Component Directory Organization	5
1.3.4. SystemVerilog Coding Guidelines	6
1.3.5. RTL Code Comments	6
1.3.6. Working with Documentation	7
2. Contribution Guide	9
2.1. OCAH Licensing Guidelines	9
2.1.1. Licensing	9
2.1.2. Third Party Code	9
2.1.3. Use of CLAs	10
2.1.4. Use of copyright messages	10
2.2. Contributing to OCAH	10
2.2.1. Getting started with the repository	10
2.2.2. Development Flow	11
2.2.3. Tracking Tasks and Bugs with GitHub	13
2.3. Managing OCAH Defects	14
2.3.1. Reporting Bugs and Defects	14
2.4. OCAH Code of Conduct	15
2.4.1. Code of Conduct	15
2.4.2. Reporting Problems	16
2.4.3. Attribution	17

The Open Chiplet Atlas Harness (OCAH) is a chiplet design IP where the non-application-specific functions are integrated to form a chiplet design framework. These non-application-specific functions include common chiplet subsystems like Security Processor (SEP), System Management Controller (SMC), System Management Network (SMN), AXI-over-UCIe bridge (AoU), Debug & Test Ports (DTP), etc.

This guide outlines the use of the repository, including licensing and guidelines for effective contribution.

WARNING

This document is currently under development. There are a number of outstanding decisions to be made.

Chapter 1. Getting Started Guide

1.1. Dependencies

WARNING

This section is currently under development. **Content has been taken verbatim from existing documentation.** More content will be added as the project matures.

This section outlines the system requirements and dependencies for the OCAH project.

System Requirements

- **Linux Distribution:** Ubuntu 20.04+ or CentOS 8+
- **Python:** Version 3.9.18 or later
- **Memory:** Minimum 16GB RAM recommended for full builds
- **Storage:** Minimum 50GB available disk space

Python Dependencies

The project uses a Python virtual environment for SystemRDL and documentation tools:

- **PeakRDL:** SystemRDL processing and RTL generation
- **Asciidoctor:** AsciiDoc documentation processing and PDF generation
- **Antora:** Multi-repository documentation site generator for AsciiDoc
- **Python packages:** Managed by the repo-level `pyproject.toml` and `uv.lock`; run `make uv-sync` or `uv sync` from the repository root.

Hardware Requirements

For simulation and synthesis:

- **CPU:** Multi-core processor (8+ cores recommended)
- **RAM:** 16GB minimum, 32GB recommended for large designs
- **Storage:** SSD recommended for build performance

License Requirements

- **Open Source Tools:** No additional licenses required
- **Commercial Tools:** Valid licenses required for VCS, Design Compiler, etc.
- **Dependencies:** Some external IPs may have specific license requirements

Environment Setup

Refer to the project setup documentation for detailed installation instructions of these dependencies.

1.2. Workflows

WARNING

This section is currently under development. **Content has been taken verbatim from existing documentation.** More content will be added as the project matures.

This document specifies the development methodology for this project.

Build with Bender and Make

Bender is used for managing IP dependencies and managing builds.

Makefiles will be used for build automation. We have previously considered other flow such as Bazel but are hesitant to take on extra complexity.

To initialise or update the Bender-generated file list, run the following command:

```
make generate_filelist
```

Simulation and Testing

Simulation with Verilator

We will use open source tools as much as possible and hence will focus on using Verilator for unit tests. Verilator has comprehensive SystemVerilog (SV) support and is widely used in industry.

Verilator ingests the RTL code along with the test code written in C/C to generate a program which is compiled and run with gcc. Writing tests in C/C has many benefits:

- high performance, typically better than commercial simulators
- incorporate third party reference code for ciphers as an example
- incorporate code for analysis
- enable development of emulation systems for firmware developers
- can be migrated to DPI based DV environments for sign-off verification

Testbench and Test Sequence

Verilator is our main simulation engine, and we have several choices on how to create unit test benches and sequences for IP components and the SEP subsystem:

- Plain *vanilla* SystemVerilog
- C/C++
- Python with cocotb

The VeeR EL2 CPU which is used in SEP has cocotb based testbenches, while Caliptra uses mostly C with Verilator. Either choice will be fine and it is *TBD* which one we shall emphasize.

We expect that chip integrators and TT verification engineers will use UVM in their verification flows. The UVM flow is however not in scope for the open source release of the OCAH project currently. We *may* release the UVM verification

infrastructure to assist teams in full chip integration efforts, but that remains TBD.

Antmicro has recently released a code coverage dashboard tool which works with Verilator. This *may* be used in the verification flows, but is TBD based on future evaluations.

The testbench is run and configured using a Python script. Run the following command to see all available options:

```
python3 tools/dv/run_dv.py --help
```

Visual Debugging and Waveform Viewer

We have two open source choices: the older GTKWave and Surfer. Surfer is the more modern of the two and runs in either the browser or standalone. It is under active development and has many nice features such as facilities for writing protocol interpreters so that we have mnemonic displays for things like opcodes instead of just raw binary or hex.

Surfer binaries are available on MacOS, Linux and Windows, but can also run in the browser which is very convenient.

References

- [Verilator documents](#)
- [cocotb](#)
- [Coverview](#)
- [Surfer project website](#)
- [RISC-V Summit paper on Surfer](#)

Linting

We use linters within the development of OCAH to perform static analysis on the codebase. The below rules will check code for errors and poor formatting. They also enforce a consistent code style.

Verilog Style and Lint checking with Verible

TODO

Verilog Style and Lint checking with Slang

TODO

TCL Script Style and Lint with Tclint

TODO

C/C++ Style and Link with Clang-Format

TODO

References

- [Verible GitHub](#)
- [Verible LSP guide](#)

1.3. OCAH development guidelines

1.3.1. Standards

The OCAH design will follow industry or FOSS standards where appropriate:

- SystemVerilog RTL
- AMBA system buses
- SystemRDL memory map
- AsciiDoc documentation with Asciidoctor

The hardware design is written in a synthesizable subset of SystemVerilog RTL language. The coding standards are adopted from the lowRISC organization and are also used in OpenTitan.

System interconnect is implemented with AMBA standard buses. AXI4 and AXI4-lite is used for data interfaces and is generally configured with 64-bit data and address. Control buses for accessing CSRs and other low speed sources are implemented with 32-bit APB.

The system memory map and component CSRs (Configuration and Status Registers) are defined in SystemRDL. This is the single source of truth and all downstream representations are rendered from this source. Examples are documentation (HTML), C headers, SystemVerilog UVM DV and SystemVerilog RTL.

1.3.2. Development Tools

We aim primarily to use open source tools for the development of OCAH in order to eliminate commercial dependencies and enabling open-sourcing not only the design, but also the associated tools.

The goal is to enable new adopters of OCAH to be able to immediately perform simulations and other analyses of the design using readily available free tools.

For internal development and testing we will employ commercial tools to validate the OCAH design, proving readiness for chip deployment i.e. tapeout. This includes:

- UVM verification
- lint tools for code quality
- logic synthesis

1.3.3. Component Directory Organization

All components will have a uniform directory structure which has the following elements:

- README.md : component overview and quick start guide in Markdown format
- doc/ : detailed AsciiDoc documentation (architecture.adoc, interface.adoc, memmap.adoc, index.adoc)

- rtl : all SystemVerilog modules <component>_*.sv
- tb : unit level Verilator tests
- regs/ : <component>.rdl SystemRDL CSR memory map definition and generated collateral
- syn : logic synthesis constraints (optional)
- lint : lint waiver files

1.3.4. SystemVerilog Coding Guidelines

The high level goals are:

- avoid SystemVerilog constructs that are error prone or not consistently supported, particularly across open source tools
- ensure that the code is tidy and easy to read
- keep the code style consistent across the project's code base regardless of who authored it

OCAH will *mostly* follow the OpenTitan SystemVerilog coding rules defined by lowRISC group in [Verilog Coding Style](#). That document is locally copied for convenience.

This section highlights a few of the most important aspects as well as a few deviations from the OpenTitan rules. If there is a conflict between the two, the specification here takes precedence.

- Use four spaces for indentation, rather than two for new files
- Use all upper case letters to name enums, constants (localparam), and parameters instead of UpperCamelCase.
- As stated in the lowRISC guidelines, SystemVerilog Interfaces can be limited in terms of tool support, so we *highly discourage* their use as they are not uniformly supported by open source tools. As an alternative, struct types can be used instead to bundle groups of input or output signals to make ports more abstract.
- Finite State Machines (FSMs) are powerful sequence control constructs but can spiral out of control quickly. Consequences are difficult to understand "next state" logic and long timing paths. Separate the combinatorial "next state" logic from the sequential state update into distinct always blocks. **Do not** combine the two into a single clocked always block. Opt for a [Moore machine](#) implementation when possible to aid in timing closure. If you have complexity of more than ten states or so, use a microsequencer approach. This is more scalable and maintainable.

There will be additional rules that apply to security sensitive components such as ciphers. These will be documented in more detail, but here is a summary for now:

- FSM state encodings are one-hot with error checking
- Error detection is implemented on storage elements and some bus and computational elements
- Secret data is transferred and stored in secret-split fashion

1.3.5. RTL Code Comments

Include a description of the purpose and overall function of the module in the header.

RTL code comments may be redundant or add little value to the code. Keep comments in the RTL as minimal as possible and, where possible, make the code self explanatory.

Do not add a comment which describes the functionality of the code where the code is clear.

If there is a tricky bit of code with some nuance, add a comment describing the overall goal or strategy of the code.

1.3.6. Working with Documentation

WARNING

This section is currently under development. **Content has been taken verbatim from existing documentation.** More content will be added as the project matures.

OCAH uses AsciiDoc for documentation:

- **Component README files:** Written in Markdown (.md) format for GitHub compatibility
- **Detailed documentation:** Written in AsciiDoc (.adoc) format for structured documentation
- **Auto-generated registers:** Generated as AsciiDoc (.adoc) files from SystemRDL sources

This approach provides:

- GitHub-native rendering for README files
- Professional documentation rendering with Asciidoctor for detailed specs
- Human readable document source text that can be revision controlled
- Multiple output formats (PDF, HTML) with cross-referencing capabilities

Asciidoctor Setup

Asciidoctor processes the AsciiDoc documentation. Install the documentation toolchain:

For Asciidoctor (Ruby gem):

```
gem install asciidoctor asciidoctor-pdf
```

For Antora (Node.js):

```
npm install -g @antora/cli @antora/site-generator-default
```

Asciidoctor is also available via the system environment:

```
apt install asciidoctor asciidoctor-pdf
```

Build Process

Generate HTML documentation:

```
cd doc && make html
```

Generate PDF documentation:

```
cd doc && make pdf
```

Diagrams

Diagrams are most easily created using the DrawIO program. Generate SVG files and embed them directly in the AsciiDoc document.

Timing diagrams are created with the Wavedrom program. The interactive application makes it easy to create the JSON source files and edit them while the CLI version of the application is used to create the output SVG files that are embedded in the AsciiDoc document.

Install Wavedrom waveform generator and viewer:

```
pip3 install wavedrom

sudo npm install -g wavedrom # for the interactive editor

sudo npm install wavedrom-cli -g # for the CLI application
```

The editor is launched with `wavedrom-editor` or directly from the website. Save the JSON source file for the figure and generate the SVG rendering as follows:

```
wavedrom-cli -i mywaveform.json5 > mywaveform.svg
```

Then embed the SVG figure into the AsciiDoc document.

References

- [Asciidoctor](#)
- [drawio](#)
- [wavedrom](#)

Chapter 2. Contribution Guide

2.1. OCAH Licensing Guidelines

2.1.1. Licensing

The main license used in this project is the Apache License, Version 2.0.

There are strict rules around the licenses used with open source RTL. OCAH will only permit incorporation of material governed under the liberal Apache 2.0 or equivalent licenses such as Solderpad.

This avoids the restrictions which come with licenses such as GPL.

The licences discussed above are documented here. Others may be added to the project over time.

- <https://www.apache.org/licenses/LICENSE-2.0>
- <https://solderpad.org/licenses/SHL-2.1/>

CAUTION

Do not attempt to commit material which was distributed under any other license without discussing first with OCAH project managers.

2.1.2. Third Party Code

The OCAH project leverages third party open source RTL code whenever practical in order to boost productivity and not "reinvent the wheel". The primary projects we reference are:

- [ETH PULP](#)
- [lowRISC OpenTitan](#)
- [OCP Caliptra](#)

This is not an exclusive list and there will inevitably be other open source projects added. We also leverage open source tools which are discussed elsewhere.

Third party components will be forked into the OCAH repository to decouple them from the originating project. We may still track the original project and update these forked versions if there is a good reason. Some reasons include incorporating bug fixes and feature enhancements.

We fully expect that third party components will be modified to some extent. This would be done by adding a wrapper around the design or portions of it, attempting to leave the core functionality unmodified. This may not always be possible, so the exact method of modification will vary.

An example modification would be that OpenTitan components use TileLink for their system bus interfaces. OCAH will use AMBA APB and AXI4, so again, these will need to be replaced.

When adopting and potentially modifying third party modules, make sure to follow all of the license rules such as keeping all attribution comments in the file headers.

2.1.3. Use of CLAs

WARNING This section needs review and agreement

A Contributor License Agreement (CLA) defines the terms under which contributions can be made to a project.

It is key to the successful functioning of an open source project such as OpenTitan. It offers critical support to three separate groups:

- Ensures that the OpenTitan project has the right to distribute the contribution.
- Confirms the contributor's ownership and grants licenses to the project.
- Provides protection for recipients and users of the contributions.

What does the project CLA cover?

WARNING This section has no content

How is the project CLA implemented?

WARNING This section has no content

2.1.4. Use of copyright messages

WARNING This section has no content

2.2. Contributing to OCAH

2.2.1. Getting started with the repository

OCAH development is hosted on GitHub under the Tenstorrent organization. Clone the public repository and use standard GitHub workflows for issues, pull requests, and code review.

Development can proceed on either macOS or Linux. Open-source tools work on both platforms; commercial EDA tools such as VCS and Design Compiler are typically available on shared Linux build hosts when needed.

Follow these steps to get set up and started with development.

1. Create a GitHub account if you do not already have one.
2. Configure Git credential management on your machine:

```
brew install git-credential-manager
```

3. Generate an SSH key pair:

```
ssh-keygen -t ed25519
```

4. Start the SSH agent and add your key. You may want to add this to your shell startup script:

```
eval "$(ssh-agent -s)"  
ssh-add <name of your key>
```

5. Add the public key to your GitHub account under <https://github.com/settings/keys>. Verify the key with:

```
ssh-add -l
```

6. Clone the repository

```
git clone git@github.com:tenstorrent/tt-oca.git  
cd tt-oca  
git checkout -b <your-branch>
```

7. Create a feature branch for your work (see below for branch management).

```
git checkout -b <your-branch>
```

2.2.2. Development Flow

We strive for a development process that balances agility and clarity. To that end, we use the following conventions and processes:

Management of Main branch

NOTE	Direct commits to main are not allowed. All changes must come through a PR. The main branch will be protected and no direct pushes are allowed.
-------------	--

- Development of each feature should happen on its own, dedicated branch
- A GitHub issue should be associated with the feature where possible, to facilitate discussion and review.
- The **main** branch is the target for all feature PRs.
- All PRs targeting **main** must pass a battery of basic smoke tests automatically run by the GitHub CI before a merge is allowed. Early in the project, there may not be meaningful tests, but these will be added as the project matures.
- The **main** branch should be kept 'working' at all times, meaning that every version should be useful for continued development and testing. Developers should be able to pull the latest **main** branch and run the code without major issues.

Management of Release branches

NOTE | Direct commits to release branches are not allowed. All changes must come through a PR.

- When a release is close to ready a release branch is created from `main` and named with the release major.patch version. This branch is used to finalize the release, e.g. by updating documentation, fixing any last minute bugs, etc.
- All changes to the release branch are based on PRs with GitHub issues targeting the release branch. If these changes also apply to `main`, then a PR should be created against those branches as well.
- All changes targeting the release branch must pass a battery of basic smoke tests automatically run by the GitHub CI before a merge is allowed as well as additional release specific tests.
- All changes to the release branch are also applied to `main`
 - This means that the release branch is not merged back into `main` when the release is finalized. A trial merge can be done to ensure that all relevant changes are applied to `main`, but this is not strictly necessary.
- When the release is finalized, the release branch is tagged with the release version.
- Minor (point) releases are made on the same major release branch, e.g. `v1.0.x`, where `x` is the minor version number. This allows users to pull from the 'v1.0.x' branch to get the latest bug fixes and minor enhancements without having to switch to a new major release branch.

Pull Requests

NOTE | All changes, however small, must go through a pull request and be reviewed before being accepted into the release codebase.

A pull request (PR in GitHub terminology or MR, merge request, in GitLab terminology) is GitHub's process of controlled merging between branches.

A pull request is a proposal to merge code changes from one branch into another (generally the main codebase). It lets developers ask team members to review, discuss, and approve updates before adding them to the released project.

Code changes not strictly related to the target feature, major change or issue should go into a separate PR. We make use of this to document changes to the codebase and facilitate code review.

A pull request (PR) should contain all the code changes required to address an issue, or all the code changes leading to the implementation of a new feature or a major change in the design. At the same time, a PR should be as small as possible to simplify and speed up the review process. It may make sense to break down larger contributions into multiple, self-contained PRs.

For bug fixes - a single PR is ideal to fix a single bug.

Continuous Integration

NOTE | All changes to the source code are tested thoroughly in a continuous integration system.

Tests run automatically when changes are proposed for inclusion by submitting a pull request, and on the master branch after changes are merged. This ensures that the source code meets certain quality criteria at all points in time, as defined by the tests which are executed.

Generated Content

Content automatically generated from source should *not* be tracked with Git. Makefiles and scripts should generate this content on-demand as part of build and simulation flows. Tracking generated content has many downsides including:

- Repository bloat from constantly changing generated content (especially from binary files).
- Unintended inconsistencies between source and generated content. For example bad code could go to tapeout because a designer forgot to run the RTL code generation script before checking in critical source script changes. Missing content is preferable to incorrect content.
- Unnecessary and confusing merge conflicts that may not be resolved properly, potentially leading to the same sort of design errors mentioned above.

There are exceptions for tracking generated content:

- Published specs and documentation generated from source for viewing directly from GitHub.

We can consider setting up [Git LFS](#) to track large binary content if necessary.

Versioning

The OCAH project will use semantic versioning (semver) version 2 for releases. The version format is **MAJOR.MINOR.PATCH**, where:

- MAJOR version when you make incompatible API changes
- MINOR version when you add functionality in a backward compatible manner
- PATCH version when you make backward compatible bug fixes

Additional labels for pre-release and build metadata are available as extensions to the MAJOR.MINOR.PATCH format.

See the documentation on [semver](#) for more details.

WARNING | We need to document how semver is applied to hardware (not yet in OpenTitan).

2.2.3. Tracking Tasks and Bugs with GitHub

Working with Issues

Development tasks are managed with GitHub issues. Issues may have parent and child issues, documenting task dependencies throughout the design.

Issues are assigned to milestones to track overall development progress, and completion of milestones may trigger design reviews or progress updates as necessary. Tags help filter and sort issues that have certain properties.

Issues can also be used to track bugs. Design, verification, and firmware engineers may file bug issues against the design. Bug issues should be assigned to the owner of the affected code, who may then reassign the issue as appropriate to manage the corrective work. More detail on raising bugs is found in the section on [Defects](#).

How to reference issues in commit messages and PRs

Commits and PRs related to existing issues or PRs should reference those. References should always be accompanied by

a brief summary of the outcome of the referenced discussion and how the current PR/commit/issue relates to that.

Who should respond to filed issues?

Anyone who feels qualified to help, though if you're working outside your usual area of expertise and making a best-effort guess it's polite to say so.

If you see an issue that might be of relevance for a particular team member, you are welcome to ping this person by commenting on the issue mentioning that person (e.g. "@githubname what do you think?").

2.3. Managing OCAH Defects

2.3.1. Reporting Bugs and Defects

NOTE

The words "Bug" and "Defect" are here used interchangeably to represent any behaviour which is not compliant to specification.

Reporting Vulnerabilities

WARNING

Do we need a Coordinated Vulnerability Disclosure (CVD) Policy?

We are dedicated to maintaining the security, integrity and reliability of our hardware and software designs, and we actively encourage responsible security vulnerability reporting from the user community.

Reporting Bugs

Ideally, all designs are bug free. However, code will be at different states of maturity. Some will still be under active testing and development. Defects may even be found in production code.

If you have discovered a possible defect in the code in this repository, please raise a new GitHub Issue to describe the bug.

Defect data

Issues raised to represent bugs should include as much detail as possible about the nature of the bug and how it can be reproduced. The goal should always be that the defect should be reproduceable from the data in the Issue.

It is helpful to include:

- A copy of, or link to, the program or simulator output.
- Any suggested resolution to the problem.
- All the information needed to reproduce the issue.
 - Git hash
 - Executed command
 - Program input
 - Simulation seed

- etc.

Reporting documentation issues

If you feel a part of the documentation is inaccurate or could be improved, please raise an Issue to specify the problem.

For document errors, it is helpful to include:

- The document in which the page number occurs.
- For a PDF document
 - The section and page numbers
- For the Web site
 - The URL and section name
- The observed error or suggested improvement in the document

2.4. OCAH Code of Conduct

WARNING | Simplified version of the OpenTitan Code of Conduct. Needs review.

2.4.1. Code of Conduct

This code of conduct applies to all spaces managed on behalf of the OCAH project.

- **Be welcoming.**

We strive to be a community that welcomes and supports people of all backgrounds and identities. This includes, but is not limited to members of any race, ethnicity, culture, national origin, colour, immigration status, social and economic class, educational level, sex, sexual orientation, gender identity and expression, age, size, family status, political belief, religion or lack thereof, and mental and physical ability.

- **Be considerate.**

Your work will be used by other people, and you in turn will depend on the work of others. Any decision you take will affect users and colleagues, and you should take those consequences into account. Remember that we're a world-wide community, so you might not be communicating in someone else's primary language. Many of our community members are here as part of their work, and content (words, images, videos) you share should be appropriate for professional workplaces.

- **Be respectful.**

Not all of us will agree all the time, but disagreement is no excuse for poor behaviour and poor manners. We might all experience some frustration now and then, but we cannot allow that frustration to turn into a personal attack. It's important to remember that a community where people feel uncomfortable or threatened is not a productive one. Members of the lowRISC community should be respectful when dealing with other members as well as with people outside the lowRISC community.

- **Be careful**

in the words and images that you choose and be kind to others. Do not insult or put down other participants.

Harassment and other exclusionary behaviour aren't acceptable. In general, if someone asks you to stop, then stop. Persisting in such behaviour after being asked to stop is considered harassment.

This includes, but is not limited to:

- Violent threats or language directed against another person.
- Discriminatory jokes and language.
- Posting sexually explicit or violent material.
- Posting (or threatening to post) other people's personally identifying information ("doxing").
- Personal insults, especially those using racist or sexist terms.
- Unwelcome sexual attention.
- Advocating for, or encouraging, any of the above behaviour.
- **When we disagree, try to understand why.**

Disagreements, both social and technical, happen all the time. It is important that we resolve disagreements and differing views constructively. Different people have different perspectives on issues. Being unable to understand why someone holds a viewpoint doesn't mean that they're wrong. Instead, focus on helping to resolve issues and learning from mistakes.

2.4.2. Reporting Problems

If you believe someone is violating the code of conduct you can report it to the Code of Conduct team by emailing XXXXXX.

NOTE

If you believe anyone is in physical danger, please notify appropriate law enforcement first. If you are unsure what law enforcement agency is appropriate, please include this in your report and we will attempt to notify them.

All reports will be kept confidential. This isn't a public list and only members of the Code of Conduct team will receive the report. If the violation occurs at an event such as a conference or workshop and requires immediate attention, you can also reach out to any of the event organisers or staff.

If you feel comfortable, please consider including:

- Your contact info (so we can get in touch with you if we need to follow up).
- Names (real, nicknames, or pseudonyms) of any individuals involved. If there were other witnesses besides you, please try to include them as well.
- When and where the incident occurred. Please be as specific as possible.
- Your account of what occurred. If there is a publicly available record (e.g. a mailing list archive or a public IRC logger) please include a link.
- Any extra context you believe existed for the incident.
- If you believe this incident is ongoing.
- Any other information you believe we should have.

2.4.3. Attribution

This text is based on the OpenTitan Code of Conduct.